

TAREA GRUPAL #2

COMUNICACIÓN ENTRE MICROCONTROLADORES

ATMEGA328P - PIC16F887

Integrantes: David Perero Navarrete, Gabriel Chacón Cárdenas, Kevin Tamay Coca, Gabriel Salas Escobar

Paralelo: 1

Grupo: 7

Nombre del Proyecto: BitBall

1. Objetivos

1.1 Objetivo General

Desarrollar un juego tipo *Breakout* utilizando dos microcontroladores, un ATmega328P y un PIC16F887, de manera que uno se encargue de la parte visual y del control del jugador, mientras que el otro gestione los efectos de sonido. Con ello se busca poner en práctica la comunicación entre microcontroladores y el manejo de periféricos en sistemas embebidos.

1.2 Objetivos Específicos

- Implementar el control de una matriz LED 8x8 mediante el ATmega328P para mostrar los elementos del juego, como la barra, la pelota y los bloques.
- Establecer una comunicación entre el ATmega328P y el PIC16F887 para que este último reproduzca sonidos de acuerdo con los eventos que ocurran durante la partida.
- Diseñar una interfaz basada en pulsadores que permita al usuario controlar el movimiento de la barra de manera sencilla y rápida.
- Incorporar tres niveles de dificultad variando la velocidad de la pelota y la cantidad de bloques, con el fin de hacer el juego más dinámico.

2. DESCRIPCIÓN TÉCNICA COMPLETA DEL JUEGO

BitBall es un juego inspirado en el clásico *Breakout*, implementado sobre una matriz LED de 8x8. El objetivo es que el jugador controle una barra para mantener la pelota en movimiento y destruir todos los bloques presentes en cada nivel. Además de la parte visual, el sistema incorpora efectos de sonido que responden a las acciones realizadas durante la partida.

Funcionamiento e interacción del usuario

El juego se controla mediante tres pulsadores:

- **Inicio del juego:** Al encender el sistema, la matriz LED muestra el símbolo de *PLAY*. Para comenzar la partida, el usuario debe presionar el botón central, lo que da inicio al primer nivel.
- **Movimiento de la barra:** Durante el juego, los botones laterales permiten mover la barra hacia la izquierda o la derecha. Al iniciar cada nivel, la pelota permanece sobre la barra sin moverse, dando al jugador la oportunidad de ubicarse antes de comenzar. La acción inicia cuando se presiona alguno de los botones de control.
- **Reinicio:** Si el jugador pierde, aparece en la pantalla el mensaje "INTÉNTALO DE NUEVO". Para volver a empezar, basta con presionar nuevamente el botón central desde la pantalla principal.

Niveles de dificultad

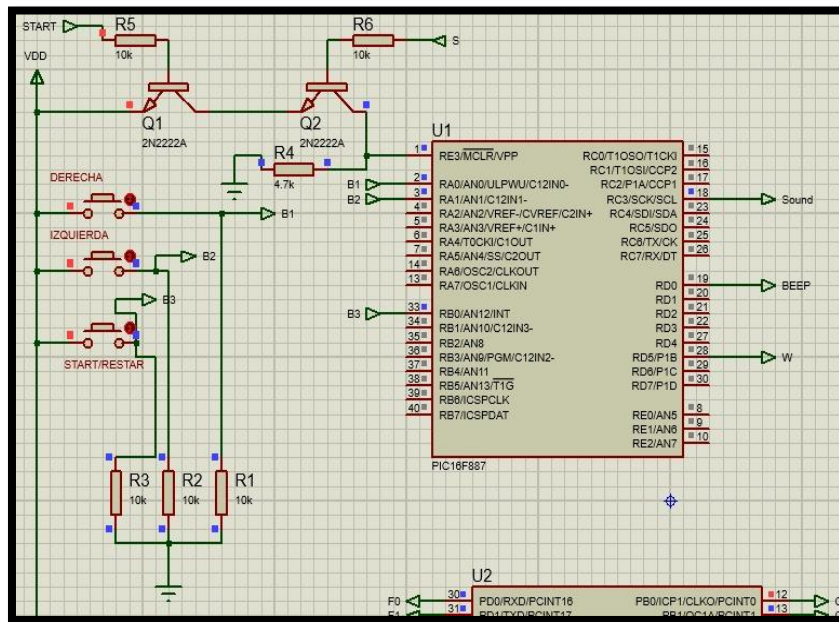
La dificultad aumenta progresivamente a medida que se avanza en el juego. Esto se logra modificando la velocidad de la pelota y la cantidad de bloques presentes en cada nivel.

- **Nivel 1 (Fácil):** Cuenta con 6 bloques y una velocidad moderada de la pelota (*limite_pelota* = 40), permitiendo que el jugador se familiarice con los controles y la mecánica del juego.
- **Nivel 2 (Medio):** Presenta 14 bloques y una velocidad ligeramente mayor, lo que exige más atención y rapidez de reacción.
- **Nivel 3 (Difícil):** Incluye 18 bloques y una velocidad más alta (*limite_pelota* = 30), aumentando considerablemente la dificultad y reduciendo el margen de error.

Condición de victoria

Cuando el jugador logra completar los tres niveles, la matriz LED muestra el símbolo de una copa como reconocimiento al haber superado todos los desafíos del juego. Esta pantalla permanece activa hasta que se presione un botón para iniciar una nueva partida.

3. Capturas de simulación de Proteus



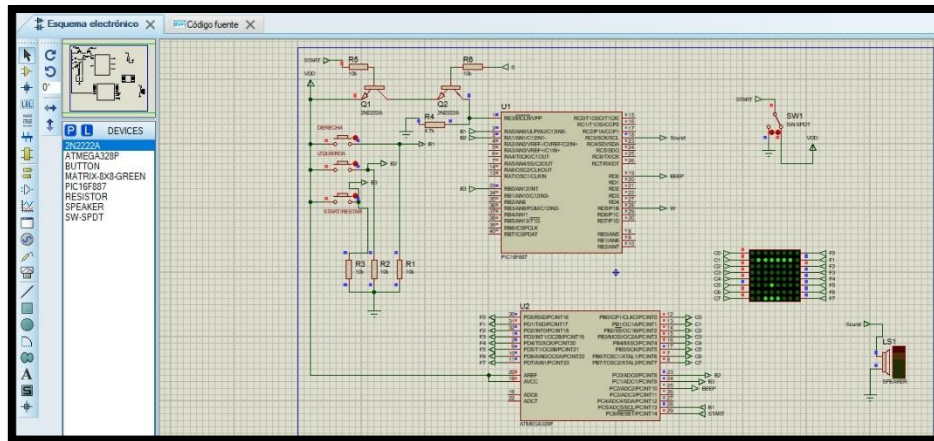


Ilustración 3: Nivel 1 del juego

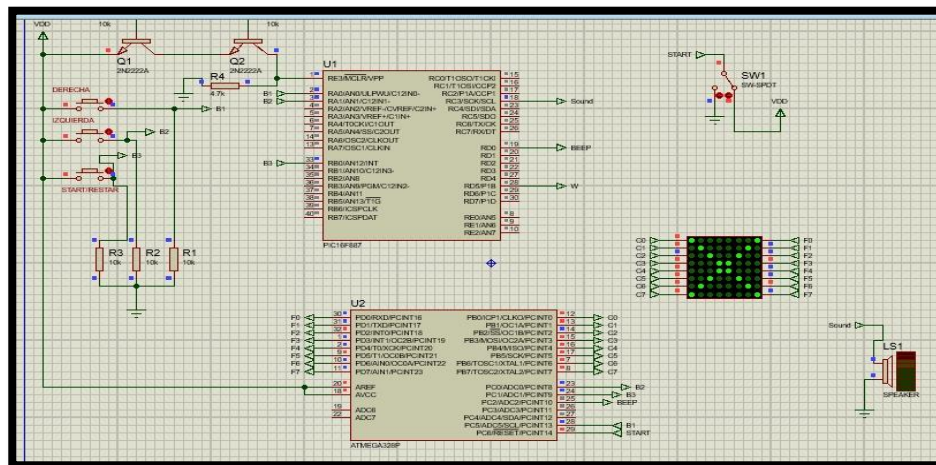


Ilustración 4: Símbolo que muestra que ha perdido el juego.

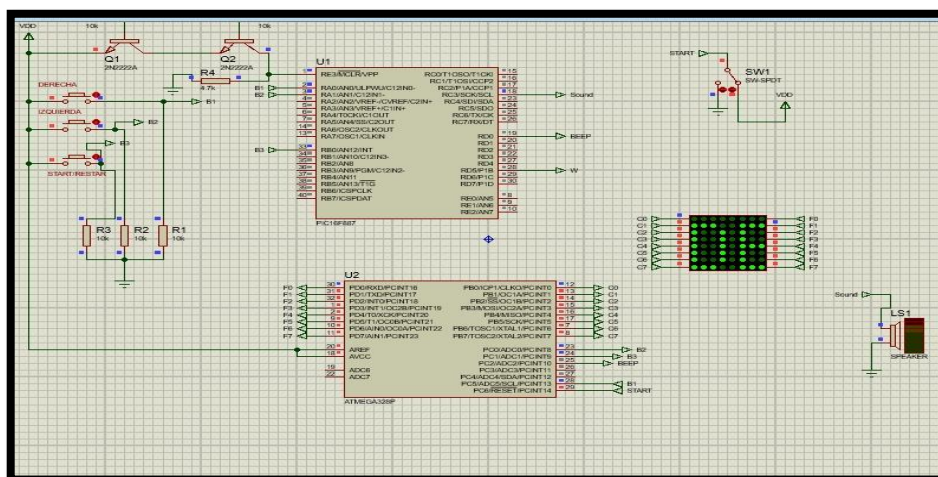


Ilustración 5: Mensaje que anima a reiniciar nuevamente el juego.

4. EXPLICACIÓN DEL CÓDIGO PARA CADA MICROCONTROLADOR (ATMEGA328P Y PIC16F887).

El proyecto fue desarrollado utilizando dos microcontroladores que trabajan de manera conjunta. Cada uno cumple una función específica dentro del sistema, lo que permite distribuir las tareas y mejorar el funcionamiento general del juego.

A. ATmega328P (Control de la matriz y lógica del juego)

El ATmega328P es el encargado de controlar la matriz LED y ejecutar toda la lógica principal del juego. Su programa funciona mediante un ciclo continuo que supervisa constantemente el estado de los botones y actualiza la información mostrada en la pantalla.

- **Gestión de entradas y estados:** Se implementó una máquina de estados para controlar las diferentes etapas del juego, como la pantalla de inicio, los niveles, la victoria y la derrota. Además, se utilizó una rutina de antirrebote por software para evitar lecturas incorrectas al presionar los pulsadores.
- **Control de la matriz LED:** La visualización se realiza mediante multiplexación. Para ello, una función recorre rápidamente las columnas de la matriz, encendiéndolas una por una. Gracias a la persistencia visual, el usuario percibe que todos los elementos permanecen encendidos al mismo tiempo.
- **Lógica del juego:** El programa calcula el movimiento de la pelota, detecta colisiones con la barra y los bloques, y actualiza el estado de la partida. Cuando todos los bloques de un nivel han sido eliminados, el sistema carga automáticamente la configuración del siguiente nivel, ajustando la velocidad de la pelota y la distribución de los obstáculos.

B. PIC16F887 (Sistema de audio)

El PIC16F887 se encarga exclusivamente de generar los efectos de sonido del juego. De esta manera, el ATmega328P puede concentrarse en la parte gráfica y en la lógica de funcionamiento.

- **Recepción de señales:** El PIC monitorea constantemente las señales enviadas por el ATmega328P. Cada combinación de entradas representa un evento diferente dentro del juego, como un rebote, la eliminación de un bloque o la victoria del jugador.
- **Generación de sonidos:** Cuando recibe una señal válida, el microcontrolador ejecuta la rutina correspondiente y genera una melodía o efecto sonoro a través del buzzer. Para ello se utilizan los temporizadores internos, que permiten controlar la frecuencia y duración de cada nota.
- **Sincronización entre sonido y juego:** La comunicación entre ambos microcontroladores se realiza de forma rápida, permitiendo que los efectos sonoros se reproduzcan prácticamente al mismo tiempo que ocurren los eventos.

visuales en la matriz LED. Esto mejora la interacción del usuario y hace que el juego resulte más dinámico.

5. DESCRIPCIÓN DEL ESQUEMA DE COMUNICACIÓN ENTRE MICROCONTROLADORES.

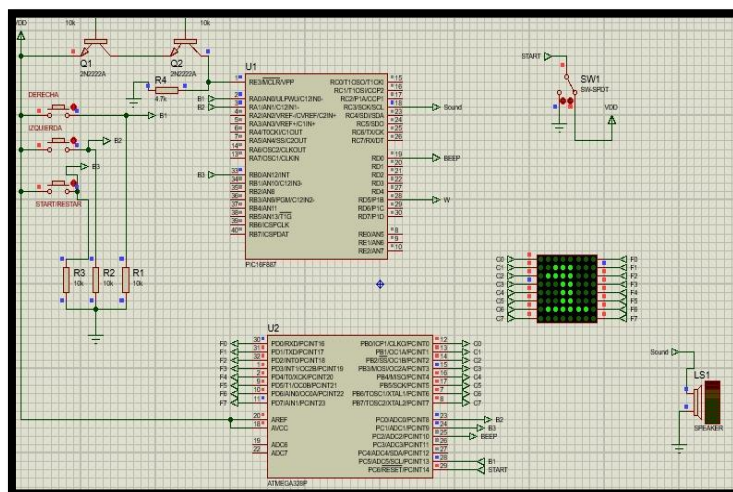


Ilustración 6: Conexión exitosa entre microcontroladores.

Para coordinar los efectos visuales del juego con los sonidos generados por el buzzer, se implementó una comunicación paralela sencilla entre el ATmega328P y el PIC16F887. Se eligió este método porque es fácil de implementar y permite transmitir información de manera rápida, logrando que los sonidos se reproduzcan casi al mismo tiempo que ocurren los eventos dentro del juego.

Funcionamiento de la comunicación

- **Líneas de datos:** Se utilizaron varios pines digitales del ATmega328P configurados como salidas, los cuales fueron conectados directamente a entradas digitales del PIC16F887. A través de estas conexiones se envían las señales necesarias para indicar cada evento del juego.
- **Identificación de eventos:** Cada combinación de bits enviada por el ATmega328P representa una acción específica. De esta forma, el PIC puede distinguir entre eventos como el inicio de la partida, la destrucción de un bloque, la victoria o la derrota, y reproducir el sonido correspondiente en cada caso.
- **Proceso de sincronización:** Cuando ocurre un evento dentro del juego, el ATmega328P genera una combinación lógica determinada en sus pines de salida. El PIC supervisa constantemente estas entradas y, al detectar un cambio válido, ejecuta la rutina encargada de generar el efecto sonoro asociado.

- **Trabajo conjunto de los microcontroladores:** Gracias a esta distribución de tareas, cada dispositivo puede concentrarse en una función específica. El ATmega328P se encarga de la lógica del juego y del control de la matriz LED, mientras que el PIC16F887 se dedica exclusivamente a la generación de sonidos. Esto permite que el sistema funcione de forma ordenada y que las respuestas visuales y sonoras se mantengan sincronizadas durante la partida.

6. ENLACE AL REPOSITORIO GITHUB:

https://github.com/DavidP121/Tarea_Comunicacion-entre-Microcontroladores

7. CONCLUSIONES

- Se logró implementar correctamente un juego interactivo utilizando un ATmega328P para el control de la matriz LED y un PIC16F887 para la generación de sonidos. La comunicación establecida entre ambos microcontroladores permitió que los efectos visuales y sonoros funcionaran de manera coordinada durante toda la partida.
- El uso de multiplexación hizo posible controlar la matriz LED de forma eficiente, mostrando correctamente la barra, la pelota y los bloques. Además, la implementación de una máquina de estados facilitó la gestión de los diferentes niveles y permitió responder adecuadamente a las acciones del jugador y a las distintas condiciones del juego.
- La simulación realizada en Proteus fue de gran ayuda para comprobar el funcionamiento general del proyecto antes de una implementación física. Gracias a esta etapa se pudo verificar que la comunicación entre los microcontroladores y el comportamiento de los periféricos fueran los esperados, permitiendo corregir posibles errores de diseño con anticipación.

8. RECOMENDACIONES

- Aunque la comunicación paralela utilizada cumplió adecuadamente con los objetivos del proyecto, sería interesante probar protocolos de comunicación como I2C o SPI en trabajos futuros. Esto permitiría reducir la cantidad de conexiones entre microcontroladores y facilitaría la incorporación de nuevos dispositivos o funciones al sistema.
- Se recomienda complementar el antirrebote implementado por software con un circuito de filtrado por hardware utilizando redes RC en los pulsadores. De esta manera se podrían minimizar posibles errores causados por los rebotes mecánicos de los botones y mejorar la confiabilidad de las lecturas.
- Como mejora para futuras versiones, se podría utilizar un controlador dedicado para la matriz LED, como el MAX7219, en lugar de realizar el multiplexado

directamente desde el microcontrolador. Esto reduciría la carga de procesamiento del ATmega328P y permitiría destinar más recursos a nuevas funciones, mejoras en la lógica del juego o la implementación de menús más elaborados.